

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

Faculty Name: Dr.G.Ayyappan

Student Reg. No:

Name:

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

*I _K.Sandhya 192424061_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1 :

Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Factor	Sequential File Organization	Heap File Organization
Data insertion	New records are inserted according to a specific ordering, so insertion can be slower because records may need to be rearranged.	Very fast because the new record can usually be placed wherever free space is available, often at the end of the file.
Data deletion	Deletion may require maintaining the sequence/order, so it can be relatively costly.	Deletion is generally faster; the record can simply be marked deleted or removed.
Data retrieval	Very efficient for sequential processing and range-based searches because records are ordered.	Efficient for retrieving all records, but slow for searching for a particular record because the file may need to be scanned.
Searching	Faster when searching according to the ordering field.	Linear search is normally required without an index.
Best suited for	Applications requiring ordered and sequential access, such as payroll processing.	Applications with frequent insertions and less frequent searches, such as temporary data storage.

Conclusion:

Sequential organization provides better ordered and range retrieval, while heap organization provides faster and simpler insertion and deletion.

QUESTION : 2.

Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Factor	Clustered Indexing	Non-Clustered Indexing
Data arrangement	The physical order of records follows the index order.	The physical order of records is independent of the index.
Number of indexes	Usually only one clustered index can exist because data can have only one physical ordering.	Multiple non-clustered indexes can be created on different columns.
Range queries	Very efficient because related records are stored close together.	Less efficient than clustered indexing for large range queries because records may be scattered.
Insertion	May be slower because the physical order may need to be maintained.	Usually easier because records can be inserted without changing the physical order.
Example	A student table clustered by Student_ID.	An index on Student_Name while records remain physically ordered by Student_ID.

Example:

```
CREATE CLUSTERED INDEX idx_student_id  
ON Student(Student_ID);
```

A non-clustered index could be created on Student_Name:

```
CREATE INDEX idx_student_name
```

ON Student(Student_Name);

Conclusion:

Clustered indexing is better for range queries and ordered retrieval, whereas non-clustered indexing provides flexibility for searching different columns.

QUESTION : 3

Compare primary indexing and secondary indexing based on storage requirements and search performance.

Factor	Primary Indexing	Secondary Indexing
Index field	Usually created on the primary key or ordering key.	Created on a non-ordering field.
Storage requirement	Generally requires less storage, especially when implemented as a sparse index.	Usually requires more storage because it is often dense.
Search performance	Very efficient because records are ordered according to the index field.	Efficient for searches on non-primary attributes but may require additional record-pointer access.
Uniqueness	Usually based on a unique key.	May be based on a non-unique attribute.
Example	Index on Student_ID.	Index on Department_ID or Student_Name.
Typical use	Finding a record using its primary key.	Searching records using alternative attributes.

Conclusion:

Primary indexing generally requires less storage and provides highly efficient primary-key searches. Secondary indexing provides additional search paths but requires more storage.

QUESTION : 4

Analyze the differences between static hashing and dynamic hashing in database systems.

Factor	Static Hashing	Dynamic Hashing
Number of buckets	Fixed when the file is created.	Can increase or decrease dynamically.
File growth	Difficult to handle when the number of records increases significantly.	Designed to handle growing and shrinking files.
Collision handling	Collisions may increase as buckets become full.	Bucket splitting helps control collisions.
Performance	Good when the estimated file size remains relatively stable.	Maintains better performance when the database size changes frequently.
Scalability	Limited scalability.	Highly scalable.
Example	A fixed hash table with 100 buckets.	Extendible hashing or linear hashing.

Conclusion:

Static hashing is simple and suitable for stable datasets, while dynamic hashing is more appropriate for databases where the number of records changes frequently.

QUESTION 5 :

Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

Factor	B-Tree	B+ Tree
Data storage	Data pointers/records can exist in internal and leaf nodes.	Actual data pointers are stored mainly in leaf nodes.

Factor	B-Tree	B+ Tree
Search	Efficient for exact-match searches.	Very efficient for both exact-match and range searches.
Range queries	Less efficient compared with B+ Tree.	Highly efficient because leaf nodes are linked.
Sequential access	Less convenient.	Very efficient because leaf nodes are linked sequentially.
Fan-out	Generally lower because internal nodes may store data.	Higher because internal nodes store only keys and pointers.
Height	Can be slightly greater.	Usually smaller for large databases.
Database usage	Less common in modern database indexing.	Widely used in database management systems.

Example:

If a database contains millions of customer records and we need all customers with IDs between 10,000 and 20,000, a B+ Tree is highly suitable because its linked leaf nodes allow efficient sequential traversal.

Conclusion:

B+ Trees are generally preferred for large-scale database applications because they provide high fan-out, efficient range searches, and efficient sequential access.

QUESTION 6 :

Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Factor	Linear Hashing	Extendible Hashing
Bucket management	Buckets are split gradually, usually in a predetermined linear order.	Buckets are split based on hash prefixes.
Directory	Does not require a directory.	Uses a directory containing pointers to buckets.
Growth	Grows incrementally as more records are inserted.	Directory can expand when bucket splitting requires more address bits.
Memory requirement	Lower because there is no directory.	Higher because a directory is maintained.
Scalability	Good for continuously growing files.	Very good and provides fast bucket access.
Complexity	Relatively simple.	More complex because of directory management.

Conclusion:

Linear hashing saves directory space and grows gradually, whereas extendible hashing provides fast access through a directory and supports efficient dynamic growth.

QUESTION 7 :

Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Factor	Dense Indexing	Sparse Indexing
Index entries	Contains an index entry for every search-key value or record.	Contains entries only for some search-key values, often one per data block.
Memory usage	Higher because many index entries are stored.	Lower because fewer entries are stored.
Search speed	Usually faster because every record can have a direct index entry.	Slightly slower because the appropriate data block must be located first.

Factor	Dense Indexing	Sparse Indexing
Storage	Requires more storage space.	Requires less storage space.
Maintenance	More expensive when records are inserted or deleted.	Easier to maintain.
Example	Index containing an entry for every student.	One index entry for each block of students.

Conclusion:

Dense indexing provides faster direct access but consumes more memory. Sparse indexing saves memory but may require additional searching within the selected data block.

QUESTION : 8

Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Factor	Indexed Sequential File Organization	Direct File Organization
Organization	Records are stored sequentially and an index provides faster access.	Records are accessed directly using a calculated address, usually through hashing.
Sequential access	Excellent.	Not as suitable.
Direct access	Good because of the index.	Very fast for exact-match searches.
Range queries	Very efficient.	Generally inefficient.
Storage overhead	Requires additional index storage.	Requires hash/bucket space and collision management.
Insertion	Can be more complex because ordering and indexes may need maintenance.	Usually fast, especially when free space is available.

Factor	Indexed Sequential File Organization	Direct File Organization
Advantages	Supports both sequential and indexed access; excellent for range queries.	Very fast exact-match retrieval and relatively simple access.
Limitations	Index maintenance requires additional processing and storage.	Poor for range queries and depends on effective hash functions.

Conclusion:

Indexed sequential organization is suitable when both sequential and range-based access are important. Direct organization is better when the main requirement is rapid exact-match retrieval.

QUESTION 9 :

Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Factor	Single-Level Indexing	Multi-Level Indexing
Index structure	One index is maintained over the data file.	An index is created over another index, forming multiple levels.
Search	May require scanning a large index.	Reduces the amount of index searching required at each level.
Memory	Requires more memory for a large single index.	More efficient for very large indexes because higher levels are smaller.
Access time	Can become slower as the index grows.	Faster for very large databases.
Complexity	Simple to implement.	More complex to maintain.
Example	One index containing keys and pointers to data blocks.	Top-level index → second-level index → data blocks.

Example structure:

Level 1 Index



Level 2 Index



Level 3 Index



Data File

Conclusion:

Single-level indexing is suitable for smaller files, while multi-level indexing is more efficient for very large databases because it reduces the number of disk accesses required to locate data.

QUESTION 10 :

Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Query Type	Hashing	Indexing
Exact-match query	Excellent performance, usually close to $O(1)$ average access.	Efficient, commonly $O(\log n)$ with tree-based indexes.
Range query	Poor because hash values do not preserve key order.	Excellent, especially with B+ Trees.
Ordering	Does not maintain ordering of keys.	Tree indexes maintain key ordering.
Collision handling	Required in hashing.	Not applicable in the same way to tree indexes.
Best use	Queries such as WHERE Student_ID = 105.	Queries such as WHERE Student_ID BETWEEN 100 AND 200.
Scalability	Dynamic hashing provides good scalability.	B+ Trees provide excellent scalability for large databases.

Example

Exact-match query:

```
SELECT *  
FROM Student  
WHERE Student_ID = 105;
```

A hash index is highly suitable because the hash function can directly identify the bucket containing Student_ID = 105.

Range query:

```
SELECT *  
FROM Student  
WHERE Student_ID BETWEEN 100 AND 200;
```

A B+ Tree index is more suitable because the keys are maintained in sorted order and the linked leaf nodes allow efficient traversal through the required range.

Overall Analysis

- Hashing → Best for exact-match queries
- B+ Tree indexing → Best for range queries
- B+ Tree → Supports both exact-match and range queries
- Hashing → Generally not suitable for range queries
- Dynamic hashing → Better when the database size changes frequently

Final conclusion:

choice between hashing and indexing depends on the query workload. For applications dominated by exact-match searches, hashing can provide very fast retrieval. For applications requiring range searches, sorting, and ordered retrieval, B+ Tree indexing is generally the better choice.